```
from google.colab import files
uploaded = files.upload()
```

```
Choose Files income-dataset.csv
income-dataset.csv(text/csv) - 3974305 bytes, last modified: 4/29/2026 - 100% done
Saving income-dataset.csv to income-dataset.csv
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

cols = ['age','workclass','fnlwgt','education','education_num',
        'marital_status','occupation','relationship','race',
        'gender','capital_gain','capital_loss','hours_per_week',
        'native_country','income']

df = pd.read_csv('income-dataset.csv', header=None, names=cols)

# Strip spaces
df = df.apply(lambda x: x.str.strip() if x.dtype == 'object' else x)

print("Dataset loaded successfully!")
print("Shape:", df.shape)
df.head()
```

```
Dataset loaded successfully!
Shape: (32561, 15)
```

	age	workclass	fnlwgt	education	education_num	marital_status	occupation	relationship	race	gender	capital_gain	c
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	
3	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	
4	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	

Next steps: Generate code with df | New interactive sheet

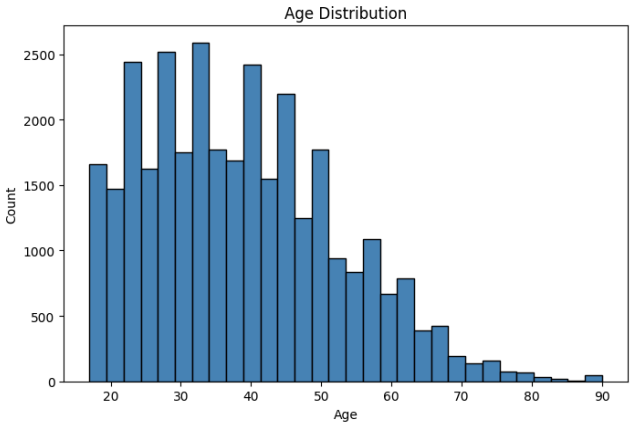
```
plt.rcParams['figure.figsize'] = (6,4)
plt.rcParams['figure.dpi'] = 100
```

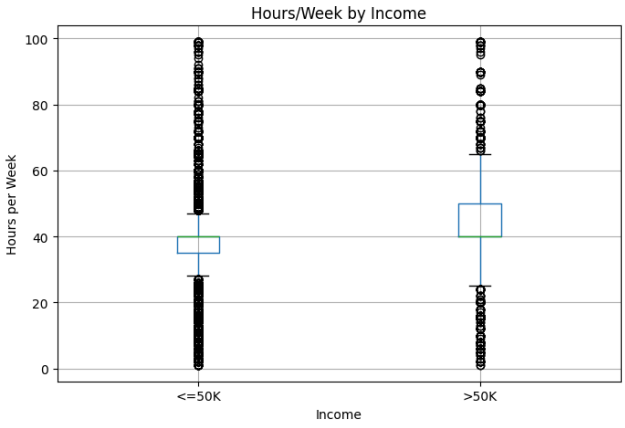
```
##Q1. What are the key attributes in the given dataset, and what types of data do they contain?

print("=== Dataset Info ===")
print(df.info())
print("\n=== Data Types ===")
print(df.dtypes)
print("\n=== Unique values per column ===")
for col in df.columns:
    print(f"{col}: {df[col].nunique()} unique | Sample: {df[col].unique()[:3]}")
```

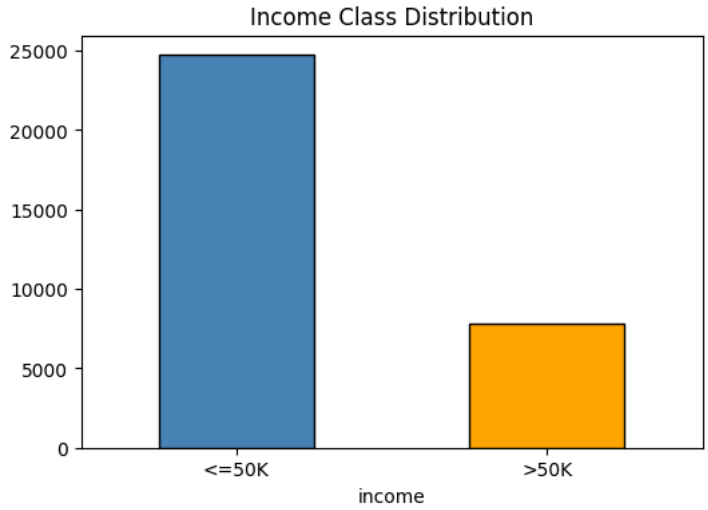
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32561 entries, 0 to 32560
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   age             32561 non-null  int64
 1   workclass       32561 non-null  object
 2   fnlwgt          32561 non-null  int64
 3   education       32561 non-null  object
 4   education_num   32561 non-null  int64
 5   marital_status  32561 non-null  object
 6   occupation      32561 non-null  object
 7   relationship    32561 non-null  object
 8   race            32561 non-null  object
 9   gender          32561 non-null  object
 10  capital_gain    32561 non-null  int64
 11  capital_loss    32561 non-null  int64

 13  native_country  32561 non-null  object
```

```
 14  income          32561 non-null  object
dtypes: int64(6), object(9)
memory usage: 3.7+ MB
None

=== Data Types ===
age                int64
workclass         object
fnlwgt             int64
education         object
education_num      int64
marital_status    object
occupation        object
relationship      object
race              object
gender            object
capital_gain       int64
capital_loss       int64
hours_per_week     int64
native_country    object
income            object
dtype: object

=== Unique values per column ===
age: 73 unique | Sample: [39 50 38]
workclass: 9 unique | Sample: ['State-gov' 'Self-emp-not-inc' 'Private']
fnlwgt: 21648 unique | Sample: [ 77516  83311 215646]
education: 16 unique | Sample: ['Bachelors' 'HS-grad' '11th']
education_num: 16 unique | Sample: [13  9  7]
marital_status: 7 unique | Sample: ['Never-married' 'Married-civ-spouse' 'Divorced']
occupation: 15 unique | Sample: ['Adm-clerical' 'Exec-managerial' 'Handlers-cleaners']
relationship: 6 unique | Sample: ['Not-in-family' 'Husband' 'Wife']
race: 5 unique | Sample: ['White' 'Black' 'Asian-Pac-Islander']
gender: 2 unique | Sample: ['Male' 'Female']
capital_gain: 119 unique | Sample: [ 2174     0 14084]
capital_loss: 92 unique | Sample: [   0 2042 1408]
hours_per_week: 94 unique | Sample: [40 13 16]
native_country: 42 unique | Sample: ['United-States' 'Cuba' 'Jamaica']
income: 2 unique | Sample: ['<=50K' '>50K']
```

```
##Q2. What is the distribution of income in the dataset? (Use histograms or boxplots for visualization)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].hist(df['age'], bins=30, color='steelblue', edgecolor='black')
axes[0].set_title('Age Distribution')
axes[0].set_xlabel('Age')
axes[0].set_ylabel('Count')

df.boxplot(column='hours_per_week', by='income', ax=axes[1])
axes[1].set_title('Hours/Week by Income')
axes[1].set_xlabel('Income')
axes[1].set_ylabel('Hours per Week')
plt.suptitle('')
plt.tight_layout()
plt.show()

df['income'].value_counts().plot(kind='bar', color=['steelblue','orange'], edgecolor='black')
plt.title('Income Class Distribution')
plt.xticks(rotation=0)
plt.show()
```





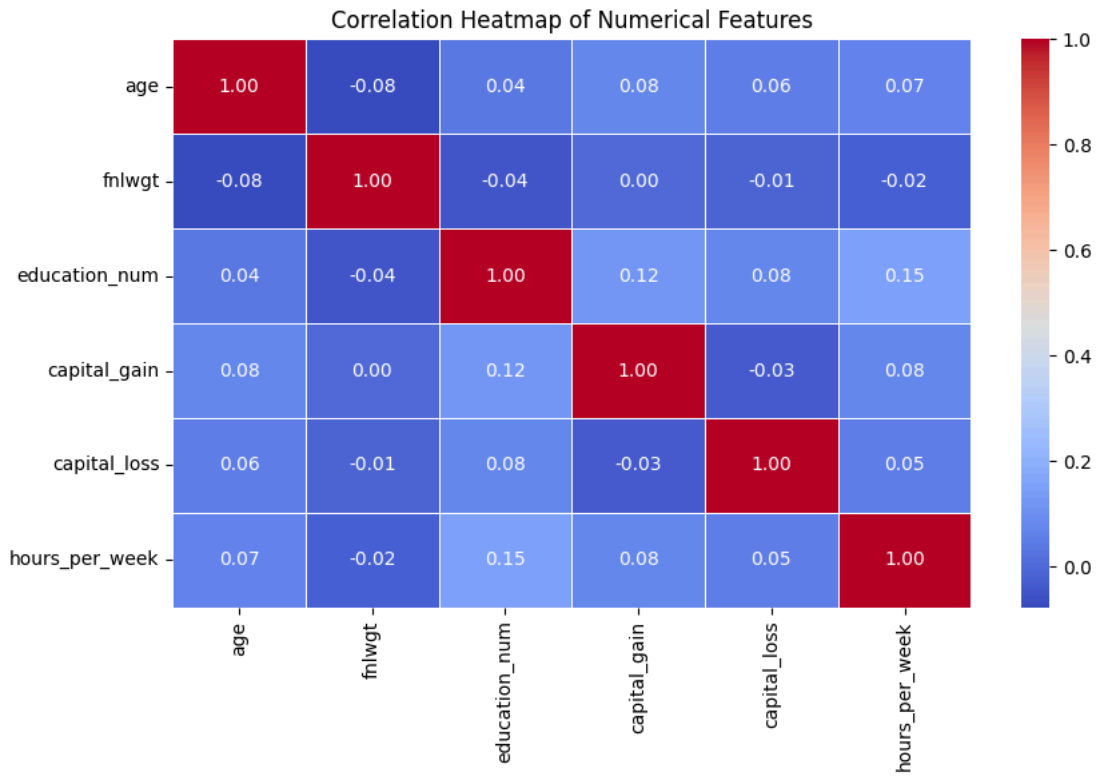



```
##Q3.  Are there any missing or null values in the dataset? How would you handle them?

df.replace('?', np.nan, inplace=True)

print("=== Null Value Count ===")
print(df.isnull().sum())

print("\n=== Percentage Missing ===")
print((df.isnull().sum() / len(df) * 100).round(2))

# Fill missing with mode
for col in ['workclass', 'occupation', 'native_country']:
    df[col].fillna(df[col].mode()[0], inplace=True)

print("\nAfter handling — nulls remaining:")
print(df.isnull().sum())
```

```
=== Null Value Count ===
age                  0
workclass         1836
fnlwgt               0
education            0
education_num        0
marital_status       0
occupation        1843
relationship         0
race                 0
gender               0
capital_gain         0
capital_loss         0
hours_per_week       0
native_country     583
income               0
dtype: int64

=== Percentage Missing ===
age               0.00
```

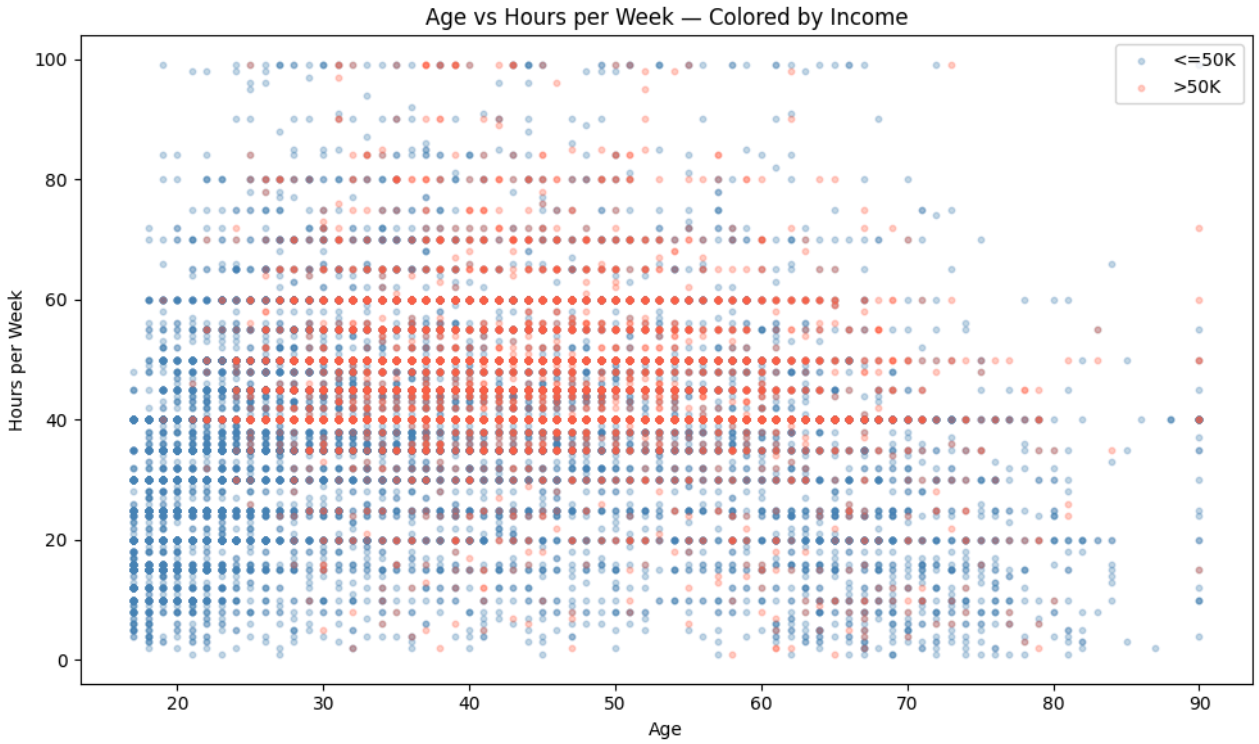
```
workclass         5.64
fnlwgt            0.00
education         0.00
education_num     0.00
marital_status    0.00
occupation        5.66
relationship      0.00
race              0.00
gender            0.00
capital_gain      0.00
capital_loss      0.00
hours_per_week    0.00
native_country    1.79
income            0.00
dtype: float64

After handling — nulls remaining:
age               0
workclass         0
fnlwgt            0
education         0
education_num     0
marital_status    0
occupation        0
relationship      0
race              0
gender            0
capital_gain      0
capital_loss      0
hours_per_week    0
native_country    0
income            0
dtype: int64
```

```
##Q4. What is the correlation between different numerical attributes? Use a heatmap to visualize this.

num_df = df[['age','fnlwgt','education_num','capital_gain',
             'capital_loss','hours_per_week']]

plt.figure(figsize=(9, 6))
sns.heatmap(num_df.corr(), annot=True, fmt='.2f',
            cmap='coolwarm', linewidths=0.5)
plt.title('Correlation Heatmap of Numerical Features')
plt.tight_layout()
plt.show()
```

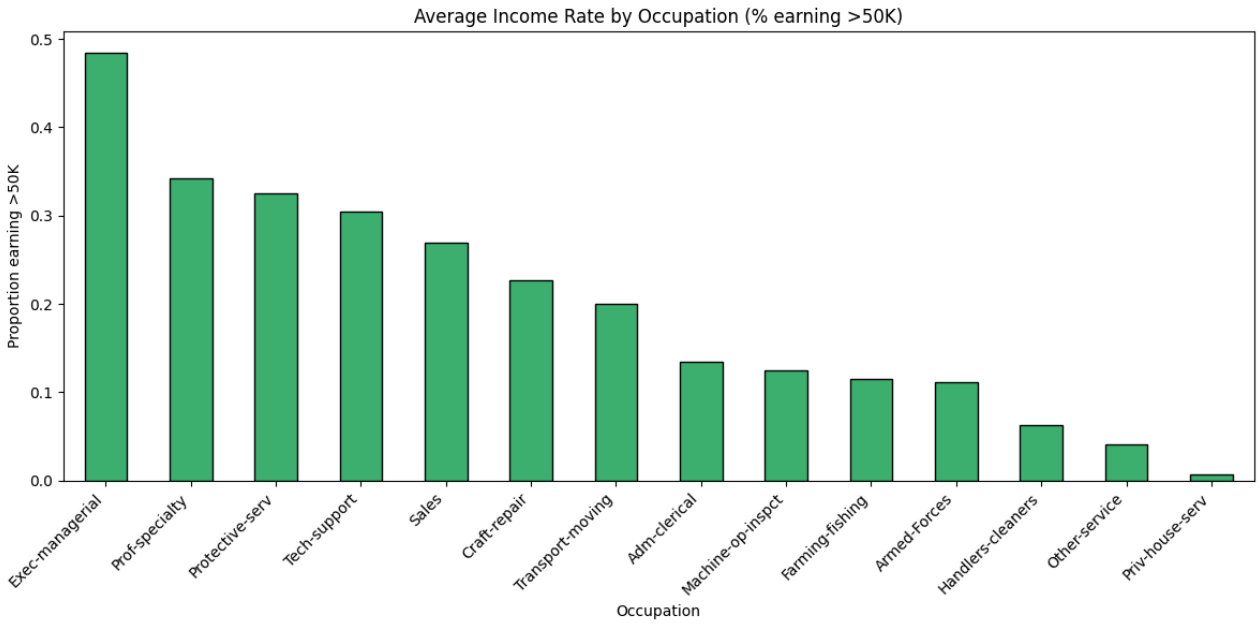



```
##Q5. How does income vary with respect to age? Generate a scatter plot to analyze the relationship.

plt.figure(figsize=(10, 6))
colors = {'<=50K': 'steelblue', '>50K': 'tomato'}
for label, group in df.groupby('income'):
    plt.scatter(group['age'], group['hours_per_week'],
```

```
                label=label, alpha=0.3, s=10, color=colors[label])

plt.title('Age vs Hours per Week — Colored by Income')
plt.xlabel('Age')
plt.ylabel('Hours per Week')
plt.legend()
plt.tight_layout()
plt.show()
```

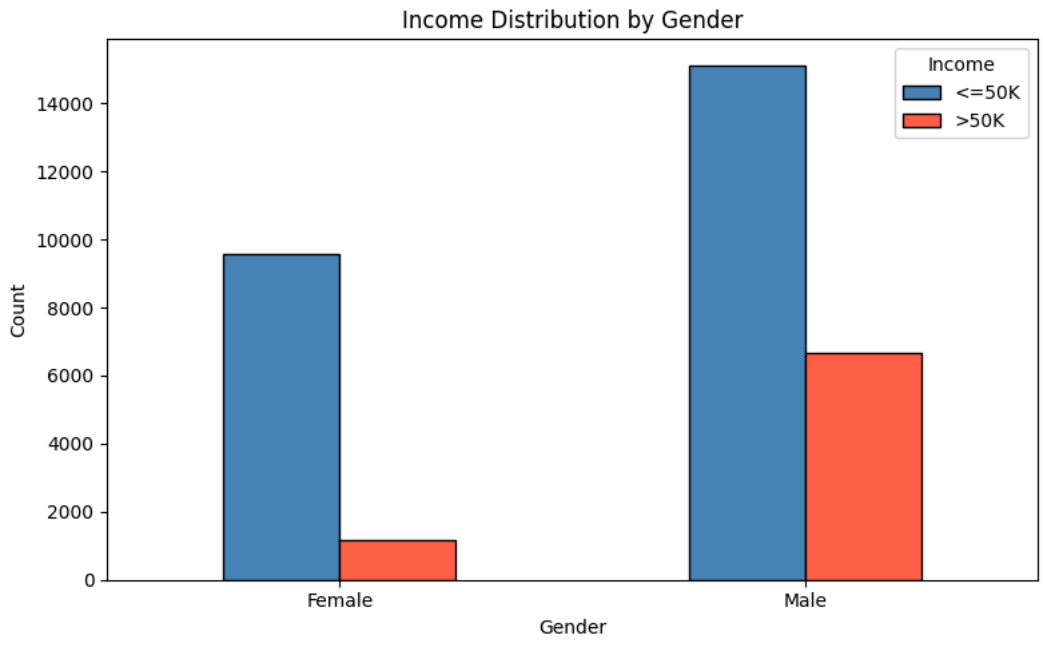



```
##Q6. Which profession has the highest and lowest average income? Visualize this using a bar chart.

df['income_binary'] = (df['income'] == '>50K').astype(int)

occ_income = df.groupby('occupation')['income_binary'].mean().sort_values(ascending=False)

plt.figure(figsize=(12, 6))
occ_income.plot(kind='bar', color='mediumseagreen', edgecolor='black')
plt.title('Average Income Rate by Occupation (% earning >50K)')
plt.xlabel('Occupation')
plt.ylabel('Proportion earning >50K')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

print("Highest avg income:", occ_income.idxmax())
print("Lowest  avg income:", occ_income.idxmin())
```



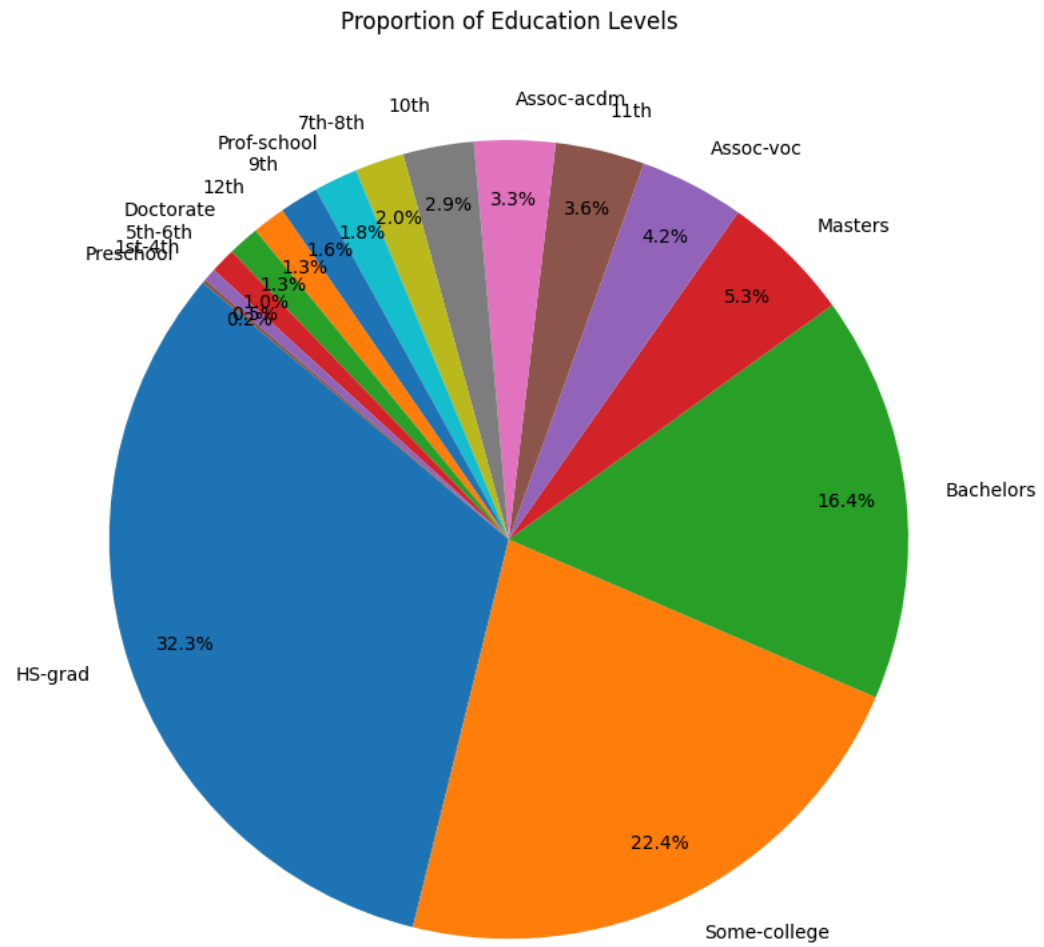

```
Highest avg income: Exec-managerial
Lowest  avg income: Priv-house-serv
```

```
##Q7. Is there any gender-based income disparity in the dataset? Use a grouped bar chart to compare.

gender_income = df.groupby(['gender','income']).size().unstack()

gender_income.plot(kind='bar', figsize=(8, 5),
                   color=['steelblue','tomato'], edgecolor='black')
plt.title('Income Distribution by Gender')
plt.xlabel('Gender')
plt.ylabel('Count')
plt.xticks(rotation=0)
plt.legend(title='Income')
plt.tight_layout()
plt.show()
```

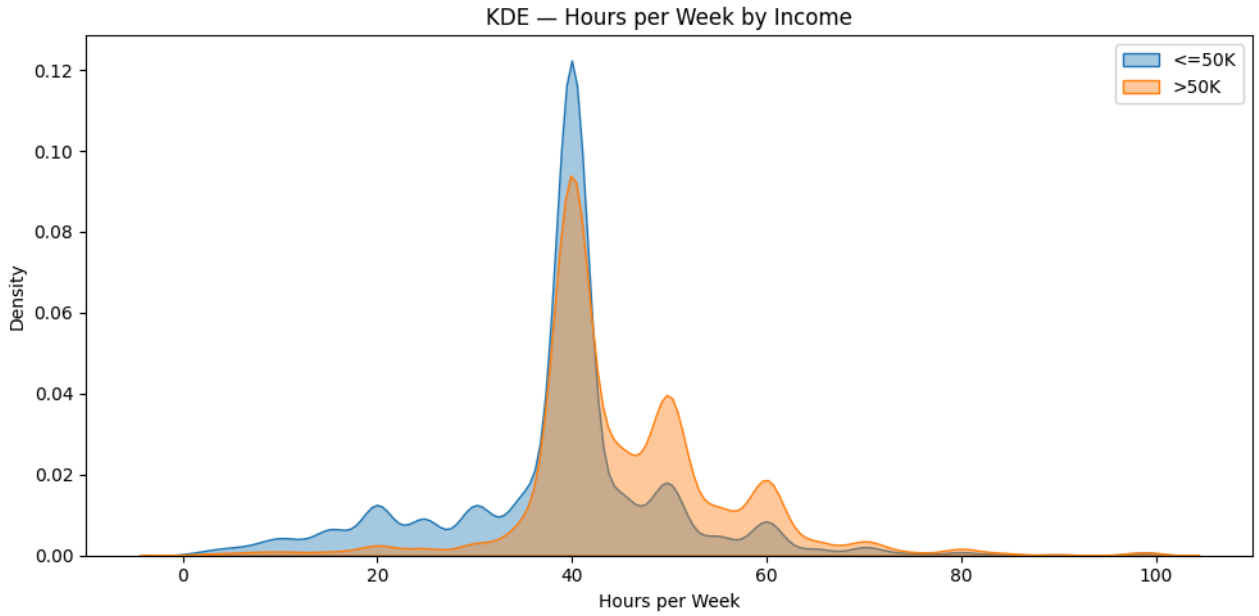



```
##Q8. Can you create a pie chart showing the proportion of different education levels in the dataset.

edu_counts = df['education'].value_counts()

plt.figure(figsize=(10, 8))
plt.pie(edu_counts, labels=edu_counts.index,
```

```
        autopct='%1.1f%%', startangle=140, pctdistance=0.85)
plt.title('Proportion of Education Levels')
plt.tight_layout()
plt.show()
```

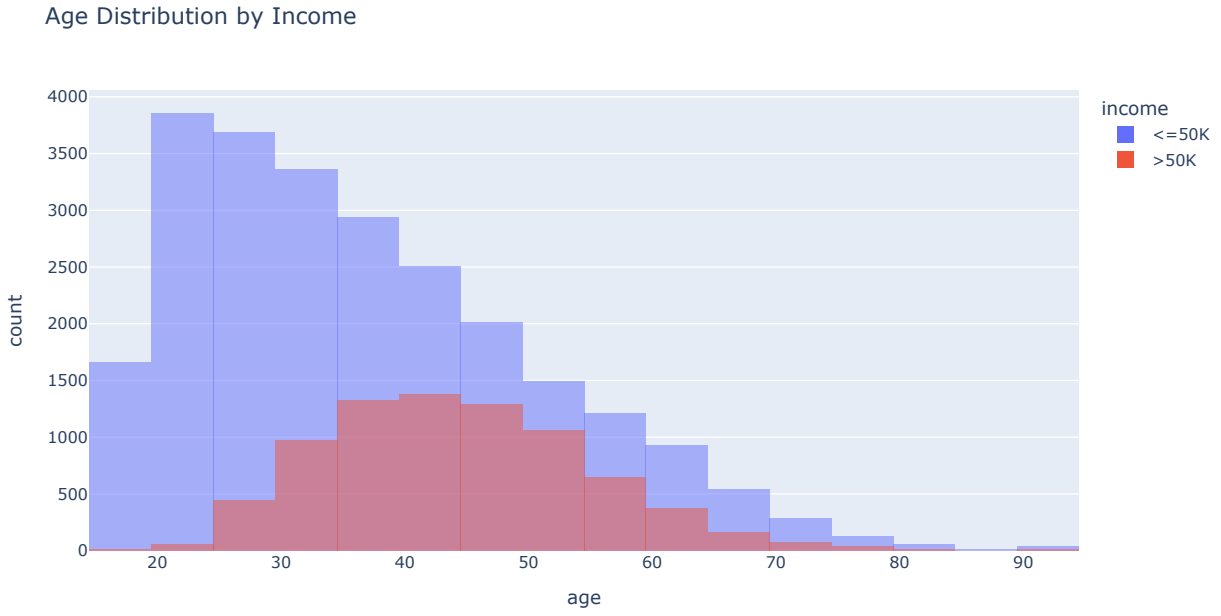


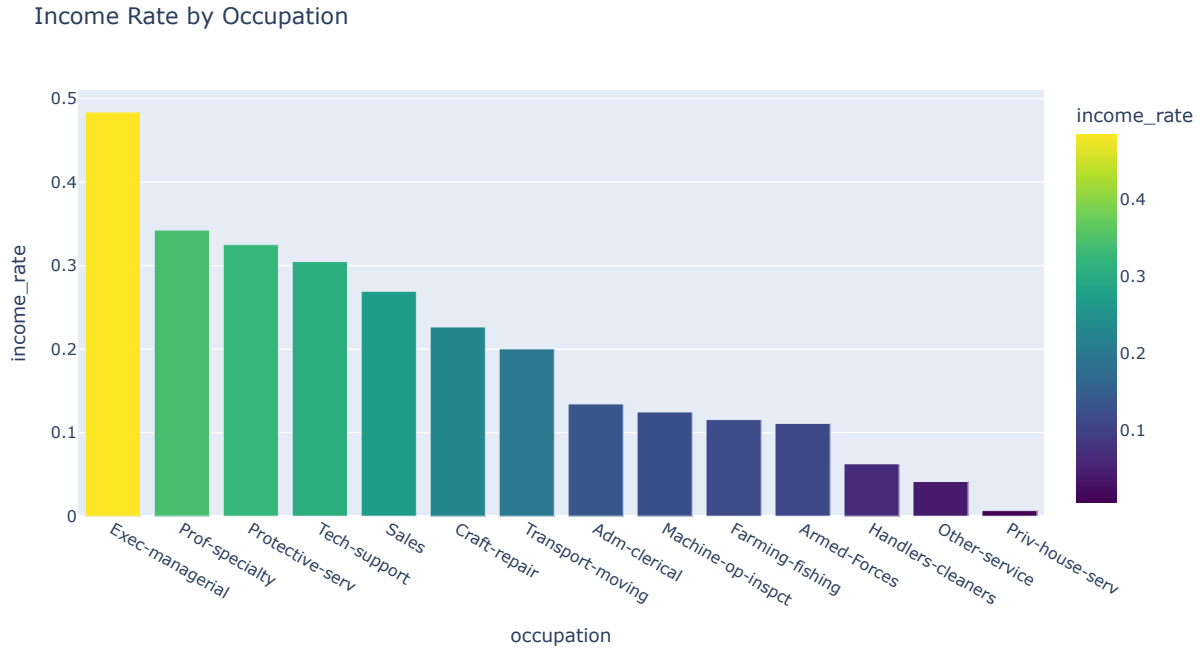

```
##Q9. Use a violin plot or KDE plot to analyze the spread of income based on work experience.

df['work_exp_group'] = pd.cut(df['hours_per_week'],
                              bins=[0,20,40,60,100],
                              labels=['Part-time','Full-time','Overtime','Heavy'])

plt.figure(figsize=(12, 6))
sns.violinplot(data=df, x='work_exp_group', y='age',
               hue='income', split=True, palette='muted')
plt.title('Age Distribution by Work Experience Group & Income')
plt.xlabel('Work Experience Group')
plt.ylabel('Age')
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 5))
for label, group in df.groupby('income'):
    sns.kdeplot(group['hours_per_week'], label=label, fill=True, alpha=0.4)
plt.title('KDE — Hours per Week by Income')
plt.xlabel('Hours per Week')
plt.legend()
plt.tight_layout()
plt.show()
```

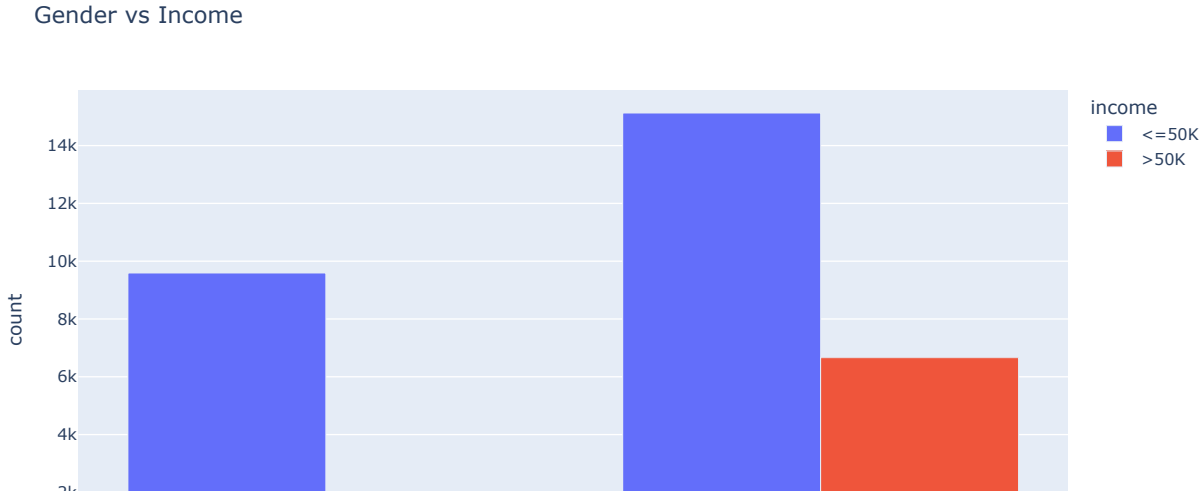



```
##Q10. Create an interactive dashboard (using tools like Tableau or Power BI) to analyze multiple variables together.

import plotly.express as px

fig1 = px.histogram(df, x='age', color='income',
                    barmode='overlay', nbins=30,
                    title='Age Distribution by Income')
fig1.show()

occ_df = occ_income.reset_index()
occ_df.columns = ['occupation','income_rate']
fig2 = px.bar(occ_df, x='occupation', y='income_rate',
              color='income_rate', color_continuous_scale='Viridis',
              title='Income Rate by Occupation')
fig2.show()

gender_df = df.groupby(['gender','income']).size().reset_index(name='count')
fig3 = px.bar(gender_df, x='gender', y='count', color='income',
              barmode='group', title='Gender vs Income')
fig3.show()

edu_df = df.groupby(['education','income']).size().reset_index(name='count')
fig4 = px.bar(edu_df, x='education', y='count', color='income',
              barmode='stack', title='Education vs Income')
```

```
fig4.show()
```


Age Distribution by Income
income
<=50K
>50K
count
age
0
500
1000
1500
2000
2500
3000
3500
4000
20
30
40
50
60
70
80
90


Income Rate by Occupation
income_rate
income_rate
occupation
0
0.1
0.2
0.3
0.4
0.5
Exec-managerial
Prof-specialty
Protective-serv
Tech-support
Sales
Craft-repair
Transport-moving
Adm-clerical
Machine-op-inspct
Farming-fishing
Armed-Forces
Handlers-cleaners
Other-service
Priv-house-serv


Gender vs Income
income
<=50K
>50K
count
2k
4k
6k
8k
10k
12k
14k